דף עזר: מבני נתונים לראיונות ג׳אווה

מבנה	סדר	סיבוכיות	מתי להשתמש	הערות ראיון
רשימות ותורים				
`int[]` / `T[]` מערך	לפי מיקום	`get` **O(1)** `find` O(n) sorted: O(log n) `add/del` **O(n)** **fixed size**	הגודל ידוע מראש. טבלה של תוצאות ביניים. ספירה לפי אות: מערך של 26 תאים.	הגודל קבוע לתמיד. `==` בודק אם זה **אותו מערך** בזיכרון, לא אם התוכן שווה. להשוואת תוכן: `Arrays.equals`. גם `Arrays.asList` מחזיר רשימה בגודל קבוע: הוספה זורקת חריגה.
`ArrayList` מערך שגדל לבד	לפי מיקום	`get` **O(1)** `find` O(n) `add/del` end O(1)∗ mid O(n)	ברירת המחדל לרשימה. קוראים הרבה לפי מיקום.	מערך מלא מוחלף במערך גדול פי 1.5. מלכודת ברשימת מספרים: `remove(1)` מוחק את מקום 1, ו־`remove(Integer.valueOf(1))` מוחק את הערך 1. מחיקה בתוך for-each זורקת `ConcurrentModificationException`. במקום: `removeIf`.
`LinkedList` רשימה מקושרת כפולה	סדר הכנסה	`get` **O(n)** `find` O(n) `add/del` ends O(1) `mem` O(n) +2 ptr	מוסיפים ומוחקים הרבה באמצע, תוך כדי מעבר על הרשימה.	כל איבר יושב במקום אחר בזיכרון ומצביע לשכנים שלו. לכן `get(i)` הולך צעד אחרי צעד. למה `ArrayList` מהיר יותר בפועל? האיברים שלו צמודים בזיכרון, והמעבד קורא זיכרון צמוד מהר.
`ArrayDeque` תור עם שני קצוות: מחסנית וגם תור	סדר הכנסה	`get` ends O(1) `find` O(n) `add/del` ends O(1)∗	מחסנית (ערימת צלחות): סוגריים, סריקה לעומק. תור (תור בקופה): סריקה לרוחב.	זה המחליף של `Stack` הישן. `Stack` יורש מ־`Vector`, ולכן: כל פעולה נועלת מנעול, ואפשר אפילו להכניס איבר לאמצע המחסנית. גם ההדפסה הפוכה: `Stack` מדפיס מלמטה למעלה, `ArrayDeque` מלמעלה למטה. אסור `null`. מחסנית: `push/pop/peek`. תור: `offer/poll/peek`.
`Priority Queue` ערימה: עץ שנשמר בתוך מערך	רק הראשון מובטח (הקטן). מעבר לא ממוין!	`get` **peek O(1)** `find` O(n) `add/del` offer/poll O(log n) remove(x) O(n)	K הגדולים ביותר. איחוד K רשימות ממוינות. מסלול קצר עם משקלים (דייקסטרה). חציון של זרם מספרים.	ברירת המחדל: הקטן בראש. הגדול בראש: `Comparator.reverseOrder()`. המשווה `(a,b)->b-a` נשבר במספרים גדולים (החיסור עובר את הגבול של `int`). במקום: `Integer.compare`. ל־K הגדולים: ערימה של K איברים בלבד, `O(n log k)`.
מפות וקבוצות (מחפשים לפי מפתח, לא לפי מיקום)				
`HashMap` מערך של מגירות: `bucket`	אין סדר	`find` **O(1) avg** **worst O(log n)** `add/del` **O(1) avg**	ספירה כמה פעמים כל דבר מופיע. Two Sum. שמירת תוצאות שכבר חושבו. קיבוץ לפי מפתח.	`hashCode` קובע לכל מפתח מספר מגירה. לכן דורסים `equals` ו־`hashCode` תמיד יחד. מפתח שמשתנה אחרי `put` לא נמצא יותר. כש־75% מהמגירות תפוסות, המערך מוכפל. מגירה עם 8 פריטים הופכת לעץ (כשיש 64 מגירות ומעלה), ולכן הגרוע הוא `O(log n)`. לא בטוח לכמה חוטים.
`LinkedHash Map` מפת גיבוב עם שרשרת לפי סדר	סדר הכנסה (או סדר השימוש)	`find` **O(1) avg** `add/del` **O(1) avg** `mem` O(n) +2 ptr	מטמון שזורק את מה שלא השתמשו בו הכי הרבה זמן (LRU). פלט באותו סדר כמו הקלט.	זה `HashMap` רגיל עם אותו מערך מגירות. כל פריט מקבל עוד שני מצביעים: לפריט שנכנס לפניו ולפריט שנכנס אחריו. כך נוצרת שרשרת לפי סדר ההכנסה. אין רשימה נפרדת: אותו פריט נמצא במגירה וגם בשרשרת. מטמון LRU: `new LinkedHashMap<>(16, 0.75f, true)` ודריסה של `removeEldestEntry`.
`TreeMap` עץ שמאזן את עצמו (אדום־שחור)	ממוין לפי מפתח	`get` first/last O(log n) `find` O(log n) `add/del` O(log n)	שאלות על טווח: כל המפתחות בין 10 ל־20. המפתח הקרוב ביותר. פגישות ולוחות זמנים.	פעולות שאין ב־`HashMap`: `floorKey` (הכי גדול שהוא קטן או שווה), `ceilingKey` (הכי קטן שהוא גדול או שווה), `headMap, tailMap`. מפתח `null` זורק חריגה. מלכודת: אם `compareTo` מחזיר 0, המפה חושבת שזה **אותו מפתח**, גם אם `equals` אומר שהם שונים.
`HashSet` **`LinkedHash Set`** קבוצה בלי כפילויות	אין סדר / סדר הכנסה	`find` **O(1) avg** `add/del` **O(1) avg**	הסרת כפילויות. ״כבר ביקרתי כאן?״. בדיקה אם משהו קיים.	בפנים זה `HashMap` שמשתמש רק במפתחות. `add` מחזיר `false` אם האיבר כבר היה: דרך קצרה לגלות כפילות.
`TreeSet` קבוצה ממוינת	ממוין	`get` first/last O(log n) `find` O(log n) `add/del` O(log n)	קבוצה שנשארת ממוינת תוך כדי שינויים. הערך הקרוב ביותר ל־x.	בפנים זה `TreeMap`. פעולות: `floor, ceiling, pollFirst`. אותה מלכודת של `compareTo`.

מבנה	סדר	סיבוכיות	מתי להשתמש	הערות ראיון
כמה חוטים בו־זמנית (חוט = `thread`, נתיב ריצה נפרד. התוכנית מריצה כמה חוטים באותו רגע, וכולם נוגעים באותו מבנה)				
`ConcurrentHashMap` מפה שמותר לשתף בין חוטים	אין סדר	`find` **`O(1) avg`** `add/del` **`O(1) avg`**	מטמון משותף. מונים שכמה חוטים מעדכנים.	ניסוי: 10 חוטים הוסיפו 1 למונה 100,000 פעמים. `HashMap` הגיע לכ־16,000, כי חוטים דרסו זה את זה. כאן, עם `merge`: בדיוק 100,000. כל כתיבה נועלת רק מגירה אחת. מלכודת: `get` ואז `put` הם שני צעדים, וחוט אחר נכנס ביניהם. במקום: `merge` או `compute`. אסור `null`.
`CopyOnWriteArrayList` מעתיקה את עצמה בכל שינוי	לפי מיקום	`get` **`O(1)`** `find` `O(n)` `add/del` **`O(n) copy`**	כמה חוטים, הרבה קריאות ומעט שינויים. למשל רשימת מאזינים לאירועים.	כל שינוי יוצר עותק חדש של כל המערך. מי שעובר על הרשימה רואה את המצב מהרגע שהתחיל, ולכן לא נזרקת חריגה.
מבנים שכותבים לבד בראיון				

`TreeNode (BST)`
עץ בינארי / עץ חיפוש
	מעבר שמאל, שורש, ימין נותן סדר ממוין	`find` `O(h)`
		`add/del` `O(h)`
		`mem` `O(n) +stack`
		`O(h)`
			שאלות רקורסיה. בדיקה אם עץ חיפוש תקין. האב המשותף של שני צמתים.	עץ חיפוש: כל מה שמשמאל קטן, כל מה שמימין גדול. אם מכניסים מספרים ממוינים, העץ נהיה שרשרת ארוכה, והחיפוש `O(n)`. לכן `TreeMap` מאזן את עצמו. בדיקת תקינות: מעבירים לכל צומת טווח מותר, לא רק משווים להורה. עץ עמוק מאוד: `StackOverflowError`.

`Trie` עץ תחיליות	לפי א״ב	`find` **`O(L)`** `add/del` **`O(L)`** `mem` `O(total chars)`	השלמה אוטומטית: מקלידים `ca` ומקבלים `cat`, `car`. בדיקת איות. חיפוש הרבה מילים בלוח אותיות.	כל קו בעץ הוא אות. מילים עם אותה התחלה חולקות דרך: `cat` ו־`car` עוברות שתיהן ב־`c` ואז `a`. סוף מילה מסומן בדגל `isEnd`. `HashSet` עונה רק על ״car היא מילה?״. `Trie` עונה גם על ״יש מילה שמתחילה ב־ca?״, בזמן `O(L)`, בלי לעבור על כל המילים.

`Graph`
רשימת שכנים או טבלה
	אין סדר	`get` `edge?` `O(1)` / `O(deg)`
		`find` `BFS/DFS` `O(V+E)`
		`add/del` `O(1)` / `O(deg)`
		`mem` `O(V+E)`
			רשתות וחברים. סדר ביצוע של משימות שתלויות זו בזו. מסלול קצר ביותר.	בג׳אווה: `List<List<Integer>>`, רשימת שכנים לכל צומת. תמיד שומרים קבוצת ״ביקרתי״, אחרת מסתובבים במעגל. מעט קשתות: רשימה. הרבה קשתות: טבלה (בדיקת קשת `O(1)`, אבל זיכרון `O(V²)`). סריקה לרוחב (BFS) מוצאת מסלול קצר רק כשלכל הקשתות אותו משקל. עם משקלים שונים: דייקסטרה.

`Union-Find` מי באותה קבוצה?	אין סדר	`find` **`O(α(n)) ≈ O(1)`** `add/del` **`union`** **`O(α(n))`**	כמה קבוצות מחוברות יש? האם קשת חדשה סוגרת מעגל?	לכל איבר יש ״בוס״. `find` מטפס עד הבוס העליון. `union` מחבר שני בוסים. שני שיפורים הופכים אותו למהיר: כל איבר מצביע ישר לבוס העליון, והקבוצה הקטנה נתלית על הגדולה. בלי שניהם: `O(n)`.

מלכודות שחוזרות בראיונות

מחלקות ישנות: כל פעולה נועלת מנעול, גם כשאין צורך. במה להשתמש במקום, לפי כמה חוטים נוגעים במבנה:

ישן	רק חוט אחד נוגע	כמה חוטים ביחד
`Hashtable`	`HashMap`	`ConcurrentHashMap`
`Vector`	`ArrayList`	`CopyOnWriteArrayList`
`Stack`	`ArrayDeque`	`ConcurrentLinkedDeque`

- `Integer` עם `==` עובד רק בין 128- ל־127. מעל זה: `false`. תמיד `equals`.
- `List.of` ו־`Map.of`: אי אפשר לשנות אותם, ו־`null` זורק חריגה.
- מיון מספרים (`int[]`) לא שומר על סדר של שווים. מיון אובייקטים שומר. שניהם `O(n log n)`.

איך בוחרים מבנה?

- מפתח ← ערך: `HashMap`. בסדר הכנסה: `LinkedHashMap`. ממוין או טווחים: `TreeMap`.
- רק ״האם קיים?״: `HashSet`. ממוין: `TreeSet`.
- גישה לפי מיקום: `ArrayList`. גודל קבוע: מערך.
- מחסנית או תור: `ArrayDeque`.
- צריך כל הזמן את הקטן או הגדול: `PriorityQueue`.
- התחלות של מילים: `Trie`. קבוצות שמתאחדות: `Union-Find`.
- כמה חוטים כותבים: `ConcurrentHashMap`.

מקרא

$O(1) < O(\alpha(n)) < O(\log n) < O(n) < O(n \log n) < O(n^2)$

- `O(1)` = מהיר תמיד, לא משנה כמה איברים. `O(n)` = עובר על הכול.
- `*` = לפעמים פעולה אחת יקרה (הגדלת מערך), אבל בממוצע `O(1)`.
- `get` גישה לפי מיקום או ראש · `find` חיפוש לפי ערך או מפתח · `add/del` הוספה ומחיקה · `mem` זיכרון.
- `avg` ממוצע · `worst` הכי גרוע · `ends` בקצוות · `mid` באמצע.
- `n` איברים · `L` אורך מילה · `V` צמתים (נקודות) · `E` קשתות (קווים) · `h` גובה העץ (במאוזן `log n`, במקרה הגרוע `n`) · `k` גודל הערימה · `deg` מספר שכנים.
- `α(n)` גדל כל כך לאט, שבפועל הוא לא עובר את 4.
- אין שורת `get` = אין גישה לפי מיקום. אין שורת `mem` = הזיכרון הוא `O(n)`.